

I. Model Architecture and Ensemble Flow

Input. Each sample is a sliding window of 100 time steps × 6 IMU channels (3 accelerometer + 3 gyroscope), flattened to a 600-dimensional vector.

Three branches. The same 600-neuron vector is fed into three parallel branches, each receiving a differently preprocessed copy of the data: raw (unscaled), standard-scaled (zero mean, unit variance), and min-max-scaled (range [0, 1]).

Encoder (identical architecture in each branch). Two dense layers compress the input into a 32-dimensional latent representation:

Dense(64, ReLU) — encoder_dense_64

Dense(32, linear) — latent_32

Classifier (identical architecture in each branch). Two dense layers map the 32-dimensional latent vector to class probabilities:

Dense(20, ReLU) — classifier_dense_20

Dense(12, Softmax) — activity_softmax

Each branch outputs a 12-dimensional probability vector over the 12 activity classes.

Concatenation. The three 12-dimensional softmax outputs are concatenated into a single 36-dimensional vector [12 | 12 | 12].

Stacked meta-classifier. A small two-layer network takes the 36-dimensional concatenated vector as input:

Input: 36

Dense(24, ReLU) — meta_dense_24

Dense(12, Softmax) — meta_softmax

Output: final 12-class prediction

Total models in the ensemble: 3 autoencoders (of which only the encoder halves are used for inference) + 3 classifiers + 1 meta-classifier = 7 Keras models, deployed as 4 flattened TFLite models.

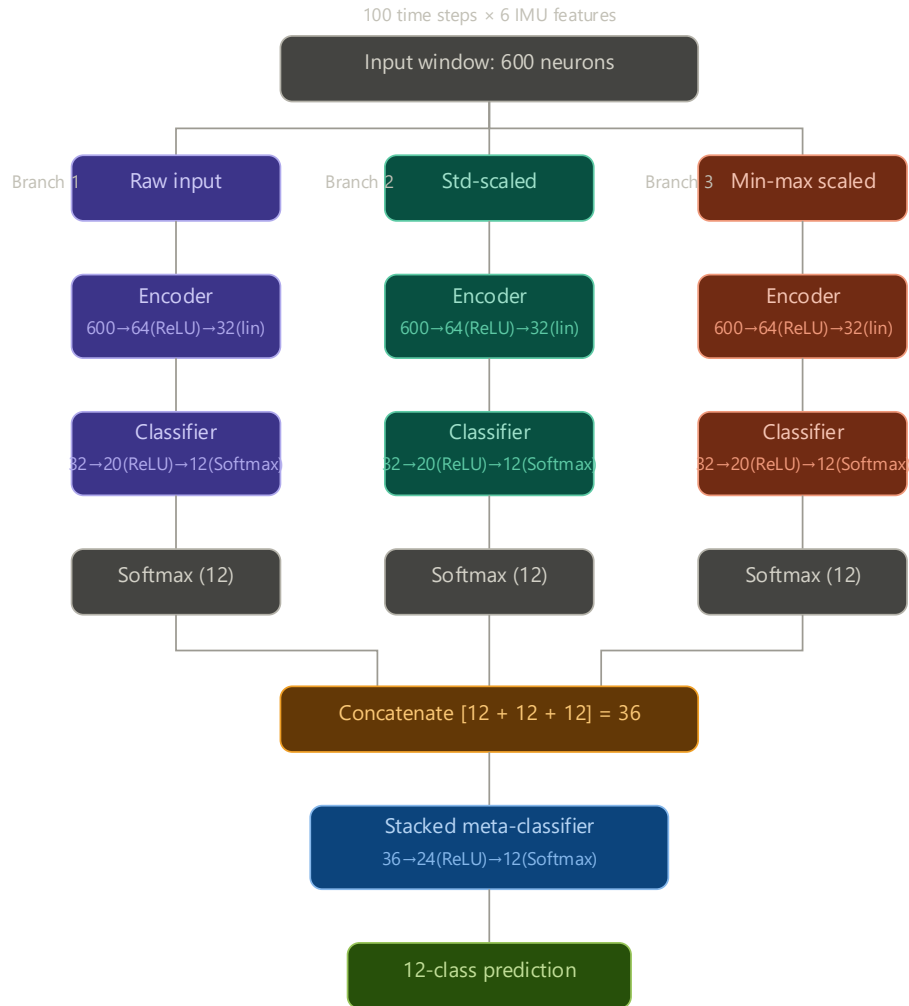


Figure 1: Visualization of the model layers, input to prediction

II. Pruning and QAT Methodology

How pruning is applied. Each flattened encoder-classifier model is wrapped with `prune_low_magnitude()` from TF-MOT. A PolynomialDecay sparsity schedule ramps from 10% to 80% sparsity between step 0 and step 500, meaning the least-magnitude weights are incrementally zeroed out over 5 training epochs. The `UpdatePruningStep` callback advances the schedule after each batch. The result is approximately 79.44% of all weights set to zero across every layer.

Why pruning wrappers are stripped before QAT. TF-MOT's pruning wrappers augment the forward pass with masking variables and override gradient updates. The `quantize_model()` API from TF-MOT cannot be stacked cleanly on top of active pruning wrappers because both tools wrap individual layers, and the nested wrapping creates incompatible layer

structures. Calling `strip_pruning()` removes the wrapper machinery and crystallizes the sparsity pattern directly into the weight tensors as literal zeros, leaving a clean standard Keras model that `quantize_model()` can then wrap.

How QAT is applied after pruning. After stripping, `quantize_model()` inserts fake quantization nodes around each Dense layer's weights and activations. During the 5 QAT training epochs, the forward pass rounds weights and activations to simulate int8 precision, then gradients flow through the rounding operation using a straight-through estimator. This forces the model to adapt its remaining non-zero weights to the noise introduced by 8-bit rounding.

Why mask enforcement is needed during QAT. When QAT fine-tunes the model, the Adam optimizer computes gradients for every weight — including the zeroed-out ones. Because a weight that is exactly zero still has a non-zero gradient at the next step, it would be updated away from zero, eroding the 79.44% sparsity the pruning step worked to achieve. A simple prune-then-QAT pipeline would silently undo the pruning. The `MaskEnforcerCallback` solves this by recording the binary sparsity mask (which weights are non-zero) immediately after stripping, then after every training batch and at the end of every epoch, multiplying each weight tensor element-wise by its mask to force pruned weights back to zero. This ensures the sparsity is preserved exactly through QAT.

int8 TFLite conversion and representative data. After QAT, `TFLiteConverter` is configured with `inference_input_type=INT8`, `inference_output_type=INT8`, and `optimizations=[DEFAULT]`. To map float32 activations to int8, the converter needs to know the practical range of values at every layer's output — this is what the representative dataset provides. Without it, the converter cannot compute the scale and zero-point parameters needed for each activation tensor's quantization. The notebook supplies a generator that yields 200 evenly-spaced training samples, which is sufficient to calibrate these ranges. The final .tflite file stores weights as int8 and uses integer-only arithmetic at inference time.

Benefits for the Arduino Nano 33 BLE Sense. The Nano 33 BLE Sense has 256 KB RAM and 1 MB flash. int8 quantization compresses each weight from 4 bytes (float32) to 1 byte, giving roughly a 4× reduction in flash footprint and reducing the working memory required to hold intermediate activations. Integer arithmetic on the Cortex-M4 is also faster and more power-efficient than floating-point, as the core can use SIMD-style integer instructions. Pruning at 79.44% sparsity means roughly 4 out of every 5 multiply-accumulate operations involve a zero weight; sparse inference libraries can skip those operations entirely, further reducing latency and energy consumption. Together, the two

techniques make it feasible to run three encoder-classifier branches plus a meta-classifier within the board's tight memory and compute budget.

III. Arduino Deployment and Real-World Behavior

This question requires you to deploy the sketch on your physical Nano 33 BLE Sense and capture a screenshot of the Serial Monitor. The written portions below can be completed based on your observations.

```
7:50:21.982 -> Raw model scores: 0.0039, 0.0000, 0.0000, 0.0039, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.9922
7:50:21.982 -> Standard-scaled model scores: 0.0000, 0.0000, 0.0000, 0.0000, 0.9336, 0.0000, 0.0000, 0.0625, 0.0000, 0.0000, 0.0000, 0.0000
7:50:22.031 -> Min-max-scaled model scores: 0.0000, 0.0000, 0.5156, 0.0000, 0.4687, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0039, 0.0000
7:50:22.031 -> Meta-classifier scores: 0.0859, 0.0508, 0.0625, 0.0000, 0.3203, 0.0117, 0.0000, 0.0117, 0.0000, 0.0117, 0.0039, 0.4375
7:50:22.031 -> -----
7:50:22.031 -> Final prediction: Jump front and back
7:50:22.031 -> Class index: 11
7:50:22.031 -> Confidence score: 0.4375
7:50:22.031 -> -----
7:50:22.525 -> Samples collected: 25
7:50:23.057 -> Samples collected: 50
7:50:23.613 -> Samples collected: 75
7:50:24.143 -> Samples collected: 100
7:50:24.143 -> Window collected. Running ensemble inference...
7:50:24.143 -> Raw model scores: 0.0039, 0.0000, 0.0000, 0.0039, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.9922
7:50:24.143 -> Standard-scaled model scores: 0.0000, 0.0000, 0.0000, 0.0000, 0.6523, 0.0000, 0.0000, 0.3437, 0.0000, 0.0000, 0.0000, 0.0000
7:50:24.188 -> Min-max-scaled model scores: 0.0000, 0.0000, 0.4258, 0.0000, 0.5664, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0039
7:50:24.188 -> Meta-classifier scores: 0.0547, 0.0352, 0.0664, 0.0039, 0.1094, 0.0156, 0.0000, 0.0312, 0.0000, 0.0352, 0.0195, 0.6250
7:50:24.188 -> -----
7:50:24.188 -> Final prediction: Jump front and back
7:50:24.188 -> Class index: 11
7:50:24.188 -> Confidence score: 0.6250
7:50:24.188 -> -----
```

Figure 2: Serial Monitor Output of the deployed model

Whether the output is acceptable at rest. If the board is flat and still, the expected prediction is one of the stationary classes — most likely "Standing still" (class 0), "Sitting and relaxing" (class 1), or "Lying down" (class 2), depending on which axis of the board is facing up. Stable, consistent predictions for one of these three classes would be acceptable. Rapidly switching predictions, or consistently predicting an active class like "Walking" or "Jogging," would indicate a problem.

Likely issues to observe. The training data came from the mHealth dataset collected from a person wearing a body-mounted sensor in a specific orientation. The Nano 33 BLE Sense placed flat on a desk will have a different gravity vector distribution than the training subject's chest-mounted sensor. The standard and min-max scaling parameters were computed from mHealth data, so if the live sensor's resting accelerometer values fall outside the training distribution (e.g., a different axis is pointing up), the scaled inputs will land in an unfamiliar region of feature space. Gyroscope noise from the Arduino sensor may also differ in magnitude from the mHealth sensor, adding further distributional shift.

Practical improvements. First, re-calibrate the scaling parameters using a short recording from the actual Arduino board rather than the mHealth statistics — this aligns the feature

distributions at inference time with those seen during training. Second, collect a small amount of labeled "resting board" data from the Arduino itself and fine-tune the classifier or at least verify the class mapping matches the board's physical orientation before deployment.

IV. Unexpected Deployment Behavior

A common symptom is the model collapsing to a single class (e.g., predicting "Climbing stairs" or "Walking" continuously regardless of what the board is actually doing). Several factors can cause this.

The most fundamental issue is dataset mismatch. The model was trained on mHealth subject 6 data, which was recorded from a sensor worn on a human body in a specific position and moving through real activities. The live Arduino board on a desk is a fundamentally different input distribution — the gravity vector projects onto different axes, the sensor noise floor may differ, and there are no dynamic motion patterns matching those in the training set.

Unit and axis orientation mismatches amplify this. The mHealth dataset records accelerometer data in m/s^2 , while the Arduino's LSM9DS1 sensor may output in g or with a different axis convention. If the Arduino sketch does not convert units to match the mHealth recordings exactly, the raw input branch receives values scaled differently than what the encoder was trained on. Even if the raw branch is off, the standard and min-max branches are also affected because the scaling parameters (mean, std, min, max) were computed from mHealth data — a live value that is numerically out of the expected range clips or saturates after scaling, producing abnormal inputs to the encoder.

Finally, the training data covers 12 specific motion patterns performed by one subject. If the board's resting orientation produces an input that most resembles one particular training class (due to any of the above shifts), the model will confidently and repeatedly predict that class because there is no "none of the above" output and no mechanism to express uncertainty.